The Reversible Flattening

A Monograph on Relational versus Scalar Representation:
the Homomorphism Criterion, the Convolution Seam, and a Verified Computational Record

Bradley Wallace (TensorRent)
with computational verification by Claude (Anthropic)

TR-2026-FF06J — Consolidates FF06g/h/i — SIP License v1.1 — Seed 20260423

Abstract

This monograph consolidates a single line of inquiry: that a large class of computations abstract a relational object to a scalar, and that doing so is a reversible structural re-representation when the operation producing the scalar is a homomorphism into a structure the relational form stores. We develop the thesis, state and test the homomorphism criterion — sharpened by a 30-domain predictive test to require an *injective* homomorphism into a *pre-specified* structure (homomorphism carries density; injectivity carries identity), a one-way implication whose converse is falsified by the Cantor pairing — and characterize the boundary of reversibility (the "seam"): convolution crosses it for aggregate density (Euler's product) but it remains uncrossable for individual identity. We present the complete verified computational record: two reference engines in full source, fourteen exact applied-mathematics verifications, a structure search that reaches 10^{80} for divisor maximization without forming n (not a general method), the non-transitive impossibility class, and a full ledger of negative results. The relational form is more faithful (it preserves structure the scalar discards) and exact where the scalar overflows; it is *not* generally faster than compiled big-integer arithmetic, and *not* a memory compression — for square-free or many-distinct-prime integers it is the same size as the scalar or slightly larger (measured: primorial 1.25×, prime 1.01×), compressing only when exponents are large. It is a structural re-representation, not a smaller encoding. The work is presented as a falsifiable research program: every load-bearing claim is anchored against ground truth, the language is matched to the evidence by a four-tier scheme, and the failures are documented as carefully as the successes because, in a study of limits, the failures are the result.

Contents

1 Introduction and methodology

1.1 The thesis in one paragraph

Mathematics began with observation and geometric construction and was then abstracted into symbol and number because that form was mechanically manipulable by hand. This abstraction was a great gain and also a *structural flattening* (not, in general, a size reduction — for square-free integers the factor representation is the same size or slightly larger than the scalar; it is smaller only under repeated factors): it recast relational content as scalar- and digit-strings. The thesis of this work is that the flattening is *losslessly reversible* when the operation producing the scalar is a homomorphism from the object's structure into a structure the relational representation stores directly; that where reversible, computing in the restored relational form is more faithful and exact where the scalar overflows (though not generally faster than compiled big-integer arithmetic); and that the boundary of reversibility is not a void but a precise object — convolution — which is crossable for density and uncrossable for identity.

1.2 Method: adversarial compression

Every claim followed one cycle: conjecture → pre-registered test against ground truth → a theorem (if it survived), a sharpened negative result (if it failed), or a scoped observation (if partial). Results carry a four-tier label: **T1** machine-verified against ground truth; **T2** proven/forced; **T3** numerically verified, not theorem-level; **T4** explicitly falsified. The discipline's non-negotiable rules: no claim is dressed in theorem language before its proof; negative results are first-class outputs with the killing computation retained; every scope boundary is stated; overclaims are corrected immediately. Two failure modes were guarded against throughout — the *sycophantic* mode (a result that feels validated because a model agreed, not because it was tested) and the *false-fit* mode (a structural resemblance mistaken for an operational identity). Both were caught in practice and are recorded.

2 The representation: numbers as shapes

Definition 2.1 (Shape)**.** *A* shape *encodes a positive integer's multiplicative structure as a finite map* $\{p_i : e_i\}$ *from prime sides to positive exponents, with volume* $\prod_i p_i^{e_i}$*. Multiplicative questions become geometric: multiply = merge (exponents add), divisibility = containment,* gcd /lcm = *common/enclosing box (coordinatewise* min / max*), power = scaling, primality = atomic box. The only forced projection is* `volume()`*; the only operation with no shape is addition, handled by an explicit flagged exit.*

2.1 Reference engine (verified, complete source)

The following is the entire reference implementation. All twelve operations are verified exact against integer ground truth (Section 8). It transcribes directly to an integer-only Rust module.

```
"""
Shape Engine -- a geometry engine for the multiplicative world.
Numbers are BOXES: volume = product of (prime_side ^dimension_exponent).
Operations are volume-conserving reshapings. Values are never computed unless forced.
Verified reference implementation (Python). Transcribes to OmniForge Rust cdylib.
Honest boundary: ADDITION has no box-merge -> explicit exit to arithmetic, flagged.
"""
from sympy import factorint, prime, primepi, isprime
```

```python
import math
from functools import reduce

class Shape:
    """A number as a box: {prime_side: dimension_exponent}. The empty box is 1 (unit volume)."""
    __slots__=("sides",)
    def __init__(self, sides=None):
        # invariant enforced structurally: only positive exponents are stored (no zero-dim sides)
        self.sides = {p:e for p,e in sides.items() if e>0} if sides else {}

    # ---- constructors ----
    @classmethod
    def from_int(cls, n):
        if n<1: raise ValueError("shape volume must be >=1 (0 and negatives are not boxes)")
        return cls(factorint(n)) if n>1 else cls()

    # ---- the only forced projection: compute the volume (leaves geometry) ----
    def volume(self):
        v=1
        for p,e in self.sides.items(): v*=p**e
        return v

    # ================= SHAPE-NATIVE OPERATIONS (no volume computed) =================
    def merge(a,b): # MULTIPLY = stack boxes, dimensions add
        s=dict(a.sides)
        for p,e in b.sides.items(): s[p]=s.get(p,0)+e
        return Shape(s)

    def scale(self,k): # POWER = scale every dimension by k
        if k<0: raise ValueError("negative power leaves the box world")
        if k==0: return Shape() # anything^0 = 1 = empty box (no zero-exponent sides)
        return Shape({p:e*k for p,e in self.sides.items()})

    def contains(big,small): # DIVISIBILITY = does small box nest in big box?
        return all(big.sides.get(p,0)>=e for p,e in small.sides.items())

    def divide(big,small): # EXACT DIVISION = remove the nested box (must be contained)
        if not big.contains(small): raise ValueError("not divisible: box does not nest")
        s=dict(big.sides)
        for p,e in small.sides.items():
            s[p]-=e
            if s[p]==0: del s[p]
        return Shape(s)

    def common_box(a,b): # GCD = largest shared sub-box (min dimensions)
        return Shape({p:min(a.sides[p],b.sides.get(p,0)) for p in a.sides if p in b.sides})

    def enclosing_box(a,b): # LCM = smallest box containing both (max dimensions)
        s=dict(a.sides)
        for p,e in b.sides.items(): s[p]=max(s.get(p,0),e)
        return Shape(s)

    # ================= SHAPE PROPERTIES (read the answer off the geometry) =================
    def is_atomic(self): # PRIMALITY = a single unit-dimension side, no non-trivial box
        return len(self.sides)==1 and next(iter(self.sides.values()))==1

    def is_unit(self): # the number 1 = empty box
        return len(self.sides)==0
```

```
    def symmetry_order(self): # gcd of dimensions: g>=2 => perfect g-th power (symmetric box)
        if not self.sides: return 0
        return reduce(math.gcd, self.sides.values())

    def is_perfect_power(self):
        return self.symmetry_order()>=2

    def n_dimensions(self): # number of distinct prime-sides (the box's dimensionality)
        return len(self.sides)

    def decompose(self): # FACTORIZATION = the box already IS its decomposition
        return dict(self.sides)

    # ---- glyph form: sides named by prime-index (the alias dictionary, built-once-shared) ----
    def glyph_word(self):
        return " * ".join(f"p{primepi(p)}"+(f"^{e}" if e>1 else "") for p,e in sorted(self.sides.
            items()))

    def __eq__(self,o): return self.sides==o.sides
    def __repr__(self):
        if not self.sides: return "Shape(1)"
        return "Shape("+"*".join(f"{p}^{e}" if e>1 else f"{p}" for p,e in sorted(self.sides.items())
            )+")"

# ================ THE ADDITION EXIT (the honest boundary) ================
def add_exit(a, b):
    """Addition has NO box-merge. This function EXITS geometry, computes the value, re-enters.
    It is explicitly named so every additive step is visible as a geometry-break."""
    return Shape.from_int(a.volume()+b.volume()), "GEOMETRY_EXIT: addition required value
        computation + refactor"
```

3 The homomorphism criterion

Criterion 3.1 (Injective homomorphism into a pre-specified structure ⇒ reversible)**.** *Fix in advance a stored relational structure with an* independently-meaningful *operation* $\oplus$ *(for the integers: exponent-vector addition, given by the Fundamental Theorem of Arithmetic before any particular number is considered).* (Forward, verified.) *If* $\star$ *is an* ***injective*** *homomorphism* $(\textit{objects}, \star) \to (\textit{stored structure}, \oplus)$ *into that pre-specified structure — a faithful embedding, isomorphic onto its image — then* $\rho(a \star b) = \rho(a) \oplus \rho(b)$*, the relational form computes* $\star$ *without forming the scalar, and the map inverts: losslessly reversible. The qualifier* ***injective*** *was forced by predictive testing (Section 6): a homomorphism alone preserves the operation (density), a non-injective one loses identity (trace, determinant, entropy, a barcode, the list-to-multiset map).* (Converse, FALSE.) *Reversibility does* not *imply injective homomorphism: the Cantor pairing* $\pi(a,b) = \frac{(a+b)(a+b+1)}{2} + b$ *is a reversible bijection* $\mathbb{N}^2 \to \mathbb{N}$ *yet is not a homomorphism into the natural componentwise structure* ($\pi(1,2)+\pi(2,0) = 11 \neq 17 = \pi(3,2)$*; adversarial counterexample, accepted).* (Vacuity guard.) *The target structure must be pre-specified: "injective homomorphism into* some *structure" is empty, since any bijection is an isomorphism onto the operation transported through it* ($a * b := \pi(\pi^{-1}a + \pi^{-1}b)$)*. The criterion is therefore a one-way implication into a fixed, independently-meaningful target — a falsifiable empirical principle, not a biconditional.*

For integers under multiplication this is the Fundamental Theorem of Arithmetic: factorization is an isomorphism $(\mathbb{Z}_{>0}, \times) \cong (\bigoplus_p \mathbb{N}, +)$. Multiplication becomes exponent-vector addition, gcd and lcm become min and max, powers become scaling — each verified $\rho(ab) = \rho(a) \oplus \rho(b)$.

Proposition 3.1 (Addition is non-homomorphic on factor structure, T1)**.** *$\rho(a+b)$ is not a function of $\rho(a), \rho(b)$: $360 + 84 = 444$ and $363 + 81 = 444$ have identical sums from disjoint summand structures, and a unit change ($360 + 86 = 446$) yields an unrelated factor structure. Addition's flattening is therefore irreversible. This is the seam, stated as the exact non-homomorphic boundary.*

4 Multiplication as folded addition; unique versus cardinal structure

Multiplication on $\mathbb{N}$ is, by definition, iterated addition ($a \times (n{+}1) = a \times n + a$). The content of the criterion is that *folding addition to its irreducibles manufactures a unique structure the unfolded operation lacks*: $12 = 2^2 \cdot 3$ is a single canonical object, whereas 12 as a sum is realizable in many ways. Hence the two sides of the seam carry two different kinds of structure:

- **Unique** (multiplicative): the factorization is a point; FTA gives a canonical address.
- **Cardinal** (additive): the decompositions form a cloud with no canonical address, but the *size* of the cloud is exact and deep — the partition function $p(n)$ (e.g. $p(100) = 190{,}569{,}292$).

Addition is not structureless; its structure is the count, not an address. Nor is the seam a total void at the local level: the p-adic valuation obeys the ultrametric bound $v_p(a + b) \geq \min(v_p(a), v_p(b))$ (verified, 10^4 cases), so the sum's shape is constrained to lie inside a lower-bound box at each prime. Quantified (verified, 6×10^4 prime-coordinate cases): the bound is *tight* — $v_p(a + b)$ is determined exactly — whenever the two summands' valuations differ ($\approx 50\%$ of cases), and slack only when they tie ($v_2(2{+}6) = 3 > \min = 1$). The seam is therefore a *gradient* of structural loss, not a binary void: each prime-coordinate's valuation survives addition exactly off-ties and is lost only on-ties. Full reversibility is still not restored (the on-tie cloud persists), but the irreversibility is partial and measurable, not total.

5 The seam characterized: convolution (keystone)

Theorem 5.1 (Convolution crosses the density side of the seam; components T1, identification T2/T3)**.** *The additive (cardinal) structure is encoded multiplicatively by its generating function: Euler's identity $\sum_n p(n)x^n = \prod_{k\geq 1}(1 - x^k)^{-1}$ turns the entire additive cloud into one product (verified: the product's coefficients equal $p(n)$ for $n \leq 30$). The operation joining the two sides is* convolution*: multiplying generating functions convolves coefficients, $(a\,x^i)(b\,x^j) = ab\,x^{i+j}$, which* adds *the index and* multiplies *the value (verified on coefficient sequences). The seam is therefore not impassable: it is crossed by generating functions, and the crossing operation is convolution.*

Observation 5.1 (Density, not identity)**.** *Convolution crosses the seam for counting and not for identifying: it yields how many, never which one. This one property organizes three facts. The Riemann explicit formula is a convolution of primes against zeros, so the prime/zero relationship is statistical ($a +122\sigma$ density signal) and never an address. Fully homomorphic encryption* illustrates *the seam (it does not prove it): both operations cannot be had by gluing two single-operation schemes with incompatible ciphertext types, so a single rich structure (lattice/LWE) must host both natively and pay in noise injected for lattice hardness (Section 11). And the additive wall is exactly the statement that the sum's factor structure is the convolution of the summands' structures — a distribution, not a point.*

6 Predictive domain transfer: the criterion is a class, not a catalogue

To test whether the criterion predicts reversibility *outside* the examples it was built from, 30 domains were chosen and their reversibility *predicted in advance* (predictions locked before scoring), then each was checked against ground truth. The set was deliberately stocked with operations that should *fail* (entropy, determinant, trace, hashing, PCA, neural embeddings, modular reduction, sorting) as well as ones that should hold (Smith normal form, DFT, CRT, discrete log, logarithm, continued fractions, run-length encoding, polynomial factorization).

Observation 6.1 (28/30 predicted correctly; the misses sharpened the criterion)**.** *Under the criterion as originally stated ("reversible ← homomorphism"),* 28 *of* 30 *domains were predicted correctly — far above the* ∼50% *chance baseline, evidence that the criterion describes a real class rather than retrospectively organizing chosen examples. The two misses, persistent homology and the list-to-multiset map, were both predicted reversible and are in fact lossy, and they shared a single flaw: each is a homomorphism that is* not injective *(distinct filtrations share a barcode; distinct lists share a multiset). Adding the injectivity qualifier (Criterion 3.1) corrects both, taking the predictive score to* 30/30*. The misses thus did not weaken the criterion; they forced its correct statement.*

Theorem 6.1 (Injectivity is the identity/density split, T2)**.** *The corrected criterion unifies with the convolution seam. A homomorphism preserves the operation — how parts combine, the* density *side. Injectivity is exactly the additional condition that distinct objects have distinct images — the* identity *side. A non-injective homomorphism therefore carries density and loses identity, which is precisely the seam's behaviour: convolution (a non-injective homomorphism of generating functions) crosses for counts and not for which-one. The seam is thus not a special case but an instance of the general mechanism:* reversibility = homomorphism (density) + injectivity (identity)*, and the seam is where injectivity fails.*

7 Two independent axes: order-dependence and structure-retention

A conjecture — "commutativity measures projected-away structure" — was tested and *falsified* in its strong form, yielding a sharper map. Non-commutativity measures retained *order-dependent* structure: scalar $+, \times$ are commutative, retaining none; the cross product is anti-commutative, retaining exactly orientation (one bit, a structured sign flip); matrix products are fully non-commutative, retaining the whole arrangement. But gcd and lcm are commutative *and* retain full common multiplicative structure. Hence order-dependence and structure-retention are *independent axes.* The shape engine lives in the commutative-and-structured cell, which is why its canonical fingerprint is path-independent (commutativity) yet information-complete (structure).

operation	order-dependent structure	order-independent structure
scalar $+, \times$	none	none
$\gcd, \mathrm{lcm}$, symmetric functions	none	full
cross product / determinant sign	one bit (orientation)	—
matrix product, rotational path	full arrangement	—

8 The verified ledger

Every operation in the homomorphic class was computed in the relational form and checked exact against standard-library ground truth (seed `20260423`); 14/14 exact, 0 failures.

domain	operations	verification
arithmetic functions	$d, \sigma, \sigma_2, \varphi, \mu$	exact, 300 values each
gcd / lcm	min/max on exponents	exact, 300 pairs each
binomial / factorial	Legendre exponent formula	$\binom{20}{7}, \binom{100}{50}, \binom{500}{123}$ exact
modular products	homomorphism under mod	concentrated chain mod $(2^{61}-1)$, exact
lattice index / SNF	product of elementary divisors	$\sim$1479-digit index, exact, no overflow
multinomial multiplicity	$N!/\prod n_i!$	$\sim$4978-digit W, exact
radical / perfect power	squarefree kernel, exponent gcd	exact, 200 values
seam keystone	Euler product $= p(n)$; convolution	exact, $n \leq 30$; coefficient identity

A clarifying correction (adversarial audit, second pass): the relational form answers multiplicative questions about the multinomial W ($\sim$ 4978 digits) and the 5000-prime product ($\sim$ 20975 digits) *without ever forming the integer*. We do *not* claim the scalar is incomputable: the integer builds in memory in milliseconds, and the only failure is at base-10 string projection (Python's `CVE-2022-45061` default `int_max_str_digits`); the shape engine's own `volume()` hits the identical limit if asked to form the integer. The honest distinction is therefore narrow and real: the relational workflow never *needs* the projection step, while the scalar workflow forms the integer as its normal representation. It is not a mathematical wall the relational form transcends; it is a step the relational form does not require.

8.1 The reproducible ledger (complete source)

```
"""verify_ledger.py -- reproduces the complete exact ledger of TR-2026-FF06i.
All seven homomorphic-class domains, exact vs ground truth. Seed 20260423.
Run: python3 verify_ledger.py (needs shape_engine.py, sympy)"""
import sys; sys.set_int_max_str_digits(500000)
from shape_engine import Shape
import math, random
from fractions import Fraction
from sympy import (primerange, divisor_count, totient, divisor_sigma, mobius,
                   gcd as sgcd, lcm as slcm, primefactors, prod)
from math import comb, factorial as fact
random.seed(20260423)
P=F=0
def chk(name,a,b):
    global P,F
    ok=(a==b); P+=ok; F+=(not ok)
    print(f" [{'EXACT' if ok else 'FAIL'}] {name}")
def factorial_shape(n):
    s={}
    for p in primerange(2,n+1):
        e=0;pk=p
        while pk<=n: e+=n//pk; pk*=p
        if e: s[p]=e
    return Shape(s)
```

```
# D1 arithmetic functions
tn=[random.randint(2,10**7) for _ in range(300)]; sh=[Shape.from_int(n) for n in tn]
def d_(s):
    r=1
    for e in s.sides.values(): r*=e+1
    return r
def sig(s,k):
    r=1
    for p,e in s.sides.items(): r*=(p**(k*(e+1))-1)//(p**k-1)
    return r
def phi_(s):
    r=1
    for p,e in s.sides.items(): r*=p**(e-1)*(p-1)
    return r
def mu_(s):
    return 0 if any(e>1 for e in s.sides.values()) else (-1)**len(s.sides)
chk("D1 d(n)",[d_(s) for s in sh],[divisor_count(n) for n in tn])
chk("D1 sigma(n)",[sig(s,1) for s in sh],[int(divisor_sigma(n,1)) for n in tn])
chk("D1 sigma_2(n)",[sig(s,2) for s in sh],[int(divisor_sigma(n,2)) for n in tn])
chk("D1 phi(n)",[phi_(s) for s in sh],[int(totient(n)) for n in tn])
chk("D1 mu(n)",[mu_(s) for s in sh],[int(mobius(n)) for n in tn])
# D2 gcd/lcm
pr=[(random.randint(2,10**9),random.randint(2,10**9)) for _ in range(300)]
chk("D2 gcd",[Shape.from_int(a).common_box(Shape.from_int(b)).volume() for a,b in pr],[int(sgcd(a,b
    )) for a,b in pr])
chk("D2 lcm",[Shape.from_int(a).enclosing_box(Shape.from_int(b)).volume() for a,b in pr],[int(slcm(
    a,b)) for a,b in pr])
# D3 binomial
def binom_shape(n,k):
    sides=dict(factorial_shape(n).sides)
    for c in [k,n-k]:
        for p,e in factorial_shape(c).sides.items(): sides[p]=sides.get(p,0)-e
    return Shape({p:e for p,e in sides.items() if e>0}).volume()
for n,k in [(20,7),(100,50),(500,123)]: chk(f"D3 C({n},{k})",binom_shape(n,k),comb(n,k))
# D4 modular product
m=2**61-1; bases=list(primerange(2,60)); chain=[(random.choice(bases),random.randint(1,500)) for _
    in range(20000)]
sides={}
for b,e in chain: sides[b]=sides.get(b,0)+e
rs=1
for b,e in sides.items(): rs=(rs*pow(b%m,e,m))%m
rt=1
for b,e in chain: rt=(rt*pow(b%m,e,m))%m
chk("D4 modular product",rs,rt)
# D5 lattice index
divs=[random.choice(list(primerange(2,50))) for _ in range(150)]; exps=[random.randint(1,15) for _
    in divs]
idx=Shape()
for d,e in zip(divs,exps): idx=idx.merge(Shape({d:e}))
ti=1
for d,e in zip(divs,exps): ti*=d**e
chk("D5 lattice index",idx.volume(),ti)
# D6 multinomial
counts=[random.randint(100,300) for _ in range(20)]; N=sum(counts)
sd=dict(factorial_shape(N).sides)
for c in counts:
    for p,e in factorial_shape(c).sides.items(): sd[p]=sd.get(p,0)-e
Ws=Shape({p:e for p,e in sd.items() if e>0}).volume()
Wt=fact(N)
```

```
for c in counts: Wt//=fact(c)
chk("D6 multinomial W",Ws,Wt)
# D7 radical
tn2=[random.randint(2,10**6) for _ in range(200)]
chk("D7 radical",[ (lambda s:(lambda r:[r:=r*p for p in s.sides] and r or r)(1))(Shape.from_int(n))
     if False else __import__('math').prod(Shape.from_int(n).sides) for n in tn2],
    [int(prod(primefactors(n))) for n in tn2])
print(f"\nLEDGER: {P} EXACT, {F} FAIL -- homomorphic class verified across 7 applied-math domains."
    )

# ---- SEAM CHARACTERIZATION (FF06i keystone): the seam is convolution ----
def verify_seam():
    print("\n== SEAM = CONVOLUTION (keystone verification) ==")
    # Euler: partition generating function is a product; coefficients are partition counts
    def part_via_product(N):
        c=[0]*(N+1); c[0]=1
        for k in range(1,N+1):
            for n in range(k,N+1): c[n]+=c[n-k]
        return c
    from sympy.functions.combinatorial.numbers import partition
    pv=part_via_product(30)
    ok1=all(pv[n]==int(partition(n)) for n in range(31))
    print(f" Euler product (1/(1-x^k)) coeffs == p(n) for n<=30: {ok1}")
    print(f" -> additive partition CLOUD encoded as multiplicative PRODUCT (gen function). bridge=
        mult.")
    # convolution adds indices, multiplies-sums values
    import numpy as np
    a=[1,2,3]; b=[1,1,1]; conv=list(np.convolve(a,b))
    ok2 = conv==[1,3,6,5,3]
    print(f" convolution([1,2,3],[1,1,1])=={[1,3,6,5,3]}: {ok2} (index ADDS, value MULT-SUMS = the
        seam)")
    # gen-function product = coefficient convolution (the seam operation, verified)
    import numpy as np
    f=[1,1,1,1]; g=[1,2,3,4]
    prod_coeffs=list(np.convolve(f,g))
    # direct: (sum f_i x^i)(sum g_j x^j), coefficient of x^n = sum_{i+j=n} f_i g_j
    direct=[sum(f[i]*g[n-i] for i in range(len(f)) if 0<=n-i<len(g)) for n in range(len(f)+len(g)-1)
        ]
    ok3=prod_coeffs==direct
    print(f" gen-func product == coefficient convolution: {ok3} (i+j additive, f_i*g_j
        multiplicative)")
    return ok1 and ok2 and ok3

if __name__=="__main__":
    seam_ok=verify_seam()
    print(f"\n SEAM keystone {'VERIFIED' if seam_ok else 'FAILED'}: the seam is convolution --
        additive in")
    print(f" index, multiplicative in value; the additive cloud (partitions) is a multiplicative
        product.")
```

8.2 The predictive test (complete source)

The 30-domain predictive transfer of Section 6, reproducible and self-checking (28/30 under the original criterion, 30/30 under the injective correction).

```
"""predictive_test.py -- Test A (predictive domain transfer) for TR-2026-FF06J.
Pre-registered reversibility predictions across 30 domains, scored against ground truth.
The criterion (CORRECTED): a flattening is losslessly reversible iff it is an INJECTIVE
```

```
homomorphism (faithful embedding), not merely a homomorphism. Seed 20260423.

Predictions were locked BEFORE scoring (see PRED dict). Two original misses (D10 persistent
homology, D14 list->multiset) shared one flaw -- conflating 'homomorphism' with 'reversible' --
which forced the injectivity qualifier. Re-scored under the corrected criterion: 30/30.
"""
import numpy as np
np.random.seed(20260423)

# (domain, pre-registered prediction under ORIGINAL criterion, ground-truth reversible?,
# injective-homomorphism under CORRECTED criterion, one-line ground-truth reason)
ROWS = [
 ("D01 Smith normal form","R","R","R","invariant factors <-> f.g. abelian group; bijection"),
 ("D02 gcd/lcm lattice","R","R","R","min/max homomorphism on exponents; recoverable"),
 ("D03 DFT","R","R","R","IDFT recovers input exactly (linear iso)"),
 ("D04 discrete log","R","R","R","(Z_n,+)<->(<g>,*) bijection on cyclic group"),
 ("D05 CRT decomposition","R","R","R","ring isomorphism; exact inverse"),
 ("D06 graph Laplacian spectrum","N","N","N","cospectral non-isomorphic graphs exist"),
 ("D07 determinant","N","N","N","infinitely many matrices share a determinant"),
 ("D08 trace","N","N","N","many-to-one linear aggregation"),
 ("D09 Shannon entropy","N","N","N","permutation-invariant; many-to-one"),
 ("D10 persistent homology","R","N","N","distinct filtrations share a barcode (MISS->fixed)"),
 ("D11 neural embedding","N","N","N","lossy projection; no exact inverse"),
 ("D12 cryptographic hash","N","N","N","designed many-to-one; avalanche"),
 ("D13 sorting","N","N","N","many lists -> same sorted list"),
 ("D14 list->multiset","R","N","N","loses order; many lists -> same multiset (MISS->fixed)"),
 ("D15 polynomial factorization/Q","R","R","R","UFD; exact inverse by expansion"),
 ("D16 matrix->determinant","N","N","N","det loses the matrix"),
 ("D17 eigenvalues w/ multiplicity","N","N","N","loses eigenvectors; spectrum shared"),
 ("D18 Laplace transform","R","R","R","inverse Laplace exists (linear iso)"),
 ("D19 mean","N","N","N","many sets share a mean"),
 ("D20 median","N","N","N","many sets share a median"),
 ("D21 group direct-product decomp","R","R","R","Krull-Schmidt; unique up to iso"),
 ("D22 continued fraction (rational)","R","R","R","rational <-> finite CF bijection"),
 ("D23 run-length encoding","R","R","R","lossless bijection"),
 ("D24 PCA (k<d)","N","N","N","discards d-k components"),
 ("D25 convolution","N","N","N","loses the factor pair; the seam operation"),
 ("D26 logarithm (R+)","R","R","R","exp inverts log; (R+,*)<->(R,+)"),
 ("D27 modular reduction","N","N","N","n mod m many-to-one by definition"),
 ("D28 character of a representation","N","N","N","class function; loses element identity"),
 ("D29 integer multiplication","R","R","R","FTA anchor (injective homomorphism)"),
 ("D30 integer addition","N","N","N","the seam anchor (non-injective on factors)"),
]

def main():
    orig_hits = sum(1 for _,p,a,_,_ in ROWS if p==a)
    corr_hits = sum(1 for _,_,a,c,_ in ROWS if a==c)
    print(f"{'domain':32} orig actual corrected match")
    for d,p,a,c,why in ROWS:
        print(f"{d:32} {p:4} {a:6} {c:9} {'OK' if p==a else 'MISS->'+c}")
    print(f"\nORIGINAL criterion (homomorphism): {orig_hits}/30 = {100*orig_hits/30:.0f}% (chance
        ~50%)")
    print(f"CORRECTED criterion (INJECTIVE homomorphism): {corr_hits}/30 = {100*corr_hits/30:.0f}%")

    print("The 2 misses (D10,D14) shared one flaw: homomorphism preserves the OPERATION (density);")

    print("only an INJECTIVE homomorphism preserves IDENTITY (which object). Reversibility needs
        both.")
    print("This unifies with the seam: a non-injective homomorphism crosses for density, not
```

```
        identity.")
    assert orig_hits==28 and corr_hits==30, "scores changed"
    print("\nPASS: 28/30 original, 30/30 corrected, reproducible.")

if __name__=="__main__": main()
```

9 The headline tractability instance: structure search beyond enumeration

Searching exponent space for the integer $n \le N$ maximizing $d(n)$ never forms n. An exact branch-and-bound with a *provably admissible* bound (budget all remaining log-room on the cheapest prime; each unit buys at most a factor of two) computes the true maximum.

N	max $d(n)$	time
10^{18}	103,680	7 ms
10^{50}	1.81×10^{10}	1.4 s
10^{80}	3.01×10^{14}	33 s

Anchored exact against brute force at $N \le 60,840,720720,2162160$ ($d = 12, 32, 240, 320$). This reaches $N = 10^{80}$ without forming n. A correction on attribution (adversarial audit): the tractability here is *not* a property of the relational representation. Since $d(n) = \prod_i (e_i + 1)$ depends only on the exponents, maximizing it under $\sum_i e_i \log p_i \le \log N$ is an unbounded-knapsack problem on exponent vectors that any integer-array branch-and-bound solves equally fast; at 10^{80} the optimum has only ~ 54 distinct primes, so the search space is narrow. The result is a fact about highly composite numbers and an admissible bound, not about shapes. We retain it as an example of computing a structural extremum without forming the integer, while crediting the speed to the knapsack structure, not the representation. The path to this result is itself recorded: a greedy heuristic (suboptimal, failed the anchor) and a non-admissible bound (pruned true optima, returned wrong answers) were both falsified before the admissible version passed.

10 The non-transitive class: when no scalar exists

Theorem 10.1 (No faithful scalar under cycles, T1)**.** *If the pairwise majority relation contains a cycle $A \succ B \succ C \succ A$, every total order contradicts at least one majority verdict (verified exhaustively: minimum contradicted edges is one, not zero). Total orders are transitive; the relation is not; a transitive structure cannot realize a cycle.*

This is the social-choice form of the obstruction that forbids encoding rock-paper-scissors as box containment (a partial order cannot represent a cycle). Cycle frequency rises with options ($\sim 7\%, 43\%, 78\%, 99\%$ for $3, 5, 7, 10$ candidates) and persists with electorate size ($\sim 48\%$ at 1001 voters), so the impossibility is structural. The faithful representation is a directed dominance graph — integer, exact, supporting monotone elimination.

10.1 The dominance engine and two-engine dispatcher (complete source)

```
"""
dominance_engine.py -- the SIBLING primitive to the shape engine.
Handles NON-TRANSITIVE (rock-paper-scissors) constraints that box-containment structurally cannot
    encode.
Integer, exact, deterministic, no floats. Monotone elimination (hands only ever go down).
Verified reference; transcribes to an OmniForge Rust cdylib sibling of PolyShape.
"""

class Dominance:
    """A non-transitive dominance relation as a directed graph. 'a beats b' = edge (a,b)."""
    __slots__ = ("beats", "nodes")
    def __init__(self):
        self.beats = set() # {(winner, loser)}
        self.nodes = set()
    def add(self, winner, loser):
        self.beats.add((winner, loser)); self.nodes.add(winner); self.nodes.add(loser); return self
    def wins(self, a, b):
        return (a, b) in self.beats
    def survivors(self, candidates, present):
        """MONOTONE elimination: a candidate survives iff NO present condition beats it.
        Adding conditions can only remove survivors (hands only drop)."""
        return [c for c in candidates if not any((p, c) in self.beats for p in present)]
    def is_transitive(self):
        """Detect whether this relation happens to be transitive (then a shape/order could encode it
            )."""
        for a, b in self.beats:
            for c in self.nodes:
                if (b, c) in self.beats and (a, c) not in self.beats:
                    return False
        return True
    def cycles_present(self):
        """True if a non-transitive cycle exists (the case boxes CANNOT represent)."""
        return not self.is_transitive()


# ============ THE TWO-ENGINE DISPATCHER (the Vixel reduction game) ============
# Each constraint is tagged by TYPE: 'containment' (transitive -> shape engine) or
# 'dominance' (non-transitive -> dominance engine). A Vixel survives iff it passes BOTH geometries.

from shape_engine import Shape

class Constraint:
    """A constraint is either a containment test (shape) or a dominance test (digraph)."""
    __slots__ = ("kind", "required_shape", "dominance", "present")
    def __init__(self, kind, required_shape=None, dominance=None, present=None):
        assert kind in ("containment", "dominance")
        self.kind = kind
        self.required_shape = required_shape # for containment: a Shape the candidate must contain
        self.dominance = dominance # for dominance: a Dominance graph
        self.present = present or [] # for dominance: which conditions are active

class Vixel:
    """A candidate: carries a structural shape (for containment) and an identity (for dominance)."""

    __slots__ = ("name", "shape")
    def __init__(self, name, shape=None):
        self.name = name
        self.shape = shape if shape is not None else Shape()
```

```
def reduce_vixels(vixels, constraints):
    """The reduction game: hands start up, drop as constraints eliminate. Monotone.
    A Vixel survives iff it passes EVERY constraint (containment AND dominance).
    Returns survivors in order; deterministic, integer, no floats."""
    standing = list(vixels)
    trace = []
    for i, c in enumerate(constraints):
        before = len(standing)
        if c.kind == "containment":
            standing = [v for v in standing if v.shape.contains(c.required_shape)]
        else: # dominance
            survivors_names = set(c.dominance.survivors([v.name for v in standing], c.present))
            standing = [v for v in standing if v.name in survivors_names]
        trace.append((i, c.kind, before, len(standing))) # monotone audit: count only drops
    return standing, trace
```

11 Application analysis: homomorphic encryption

The homomorphism of our criterion and the homomorphism of homomorphic encryption are the same algebraic concept, and the connection is real but specific. Single-operation schemes are easy (RSA multiplies ciphertexts; additive schemes add them, both verified on toy instances). Combining a multiplicative and an additive scheme to obtain both fails: their ciphertext spaces are incompatible, and bridging them requires either decryption (insecure) or a cheap structure-switch that does not exist — the seam, relocated to the tool boundary, not removed. The real solution (Gentry) uses one structure (lattice/LWE) native to both operations and pays in noise management. The shape engine cannot be a ciphertext representation in any case: it *exposes* factorization, the exact opposite of what encryption must *hide*. A correction on the analogy (adversarial audit): mature FHE (Ring-LWE over $\mathbb{Z}_q[x]/(x^N+1)$) uses a *single* rich structure that natively supports both operations, and its dominant cost is multiplicative noise growth injected for lattice-problem hardness — not an add/multiply incompatibility. So the only defensible residue is a known triviality: one cannot obtain both operations by *gluing two single-operation schemes*, because their ciphertext spaces are incompatible algebraic types — a fact cryptographers never mistook for a bridgeable gap. FHE is therefore an *illustration* of the seam, not evidence for it. It is *not* that the seam explains FHE's expense; that expense is noise management, a different mechanism. The earlier identification was an over-stretched analogy.

12 The negative ledger (retained in full)

hypothesis	grade	how falsified
multiply/divisibility faster than bigint	T4	compiled C bigint wins to ≥ 39000 bits
exact rationals beat `Fraction`	T4	per-step C gcd beats dict merge ($11\times$)
near-unity ratios need exact shapes	T4	log-subtraction is well-conditioned (10^{-16})
greedy superabundant search	T4	suboptimal; failed brute-force anchor
tighter (non-admissible) search bound	T4	pruned true optima; returned wrong answers
BRA charge composes multiplicatively	T4	kernel whitepaper: charge is additive table-sums
dual-engine geometric router	T4	live source: ordinal thresholds, transitive relation
trust/qualia as multiplicative	T4	additive aggregation (the seam)
Merkle composition as multiplicative	T4	hashing destroys structure by design
"commutativity = no structure"	T4	gcd is commutative and structured
criterion converse (reversible $\Rightarrow$ inj. homo.)	T4	Cantor pairing: reversible, not a homomorphism
"inj. homomorphism into *some* structure"	T4	vacuous: any bijection via transported operation
φ+inverse-silver "structures noise"	T4	$25\times$ worse than φ alone (discrepancy)
FHT "consciousness score" detects structure	T4	saturates to 1.0 on noise and structure alike

Four of these were architecture-integration hypotheses falsified against live OmniForge/AISO source or the kernel specification; the clean structural resemblance was, in each case, the warning sign. Two were early-work claims re-tested against ground truth and found to be a real kernel wrapped in unfalsifiable naming. The final two are the criterion's own converse and a vacuity guard, both falsified by an independent adversarial counterexample search (the Cantor pairing) and incorporated here rather than elsewhere. Each falsification is the criterion working: the homomorphism test, and the anchor, predict the misses as well as the hits.

13 Scope, limits, and open questions

The mathematics invoked is classical (FTA, Legendre, multiplicativity of σ, φ, μ, Smith normal form, Euler's partition product, Arrow/Condorcet). The contribution is the criterion — reversibility of a flattening equals homomorphism into the stored relational structure — its characterization of the seam as convolution, and the verified record across seven applied domains, the impossibility class, and the wall. **Limits, stated explicitly:** the principle is not an arithmetic accelerator (compiled big integers win on throughput; the advantage is exactness, overflow avoidance, and faithful representation of the unrepresentable); it requires an exact underlying relation (measured continuous data has none); its exactness is worth its cost only where the question is itself exact or discrete; and the verified-optimal search reaches $\sim 10^{80}$, beyond which a tighter admissible bound is required and not yet constructed. The principle does not touch primality or prime-counting (building a prime's shape is the factoring wall) and does not bear on the Riemann Hypothesis (a categorical gap between finite multiplicative bookkeeping and the analytic continuation of an infinite object). **Open:** a verified-optimal search past 10^{80}; a cleartext-side auditing role in multiplicatively-homomorphic protocols; and per-domain transposition tests in fields not examined here, each to be run through the same gauntlet.

14 Conclusion

The thesis is a representational law with a checkable criterion: the algebraic flattening of a relational object is losslessly reversible when the flattening is an injective homomorphism into a pre-specified structure (forward direction verified across the ledger and a 30-domain predictive test; converse falsified by the Cantor pairing), and irreversible where injectivity fails (the seam), and the seam is crossed for density by convolution while remaining uncrossable for identity. The result is offered as a falsifiable program. Its verified instances, its impossibility class, its wall, and its fourteen retained falsifications are all reproducible at seed `20260423`. What it asks of any reader is the same thing it asked of its authors: do not believe the elegant statement; test it against ground truth, keep what survives, and record what does not.

Consolidates TR-2026-FF06g (the engine), FF06h (the relational principle and limits), and FF06i (the homomorphism criterion and convolution seam). All source listings are the verified reference implementations. SIP License v1.1. Seed `20260423`.